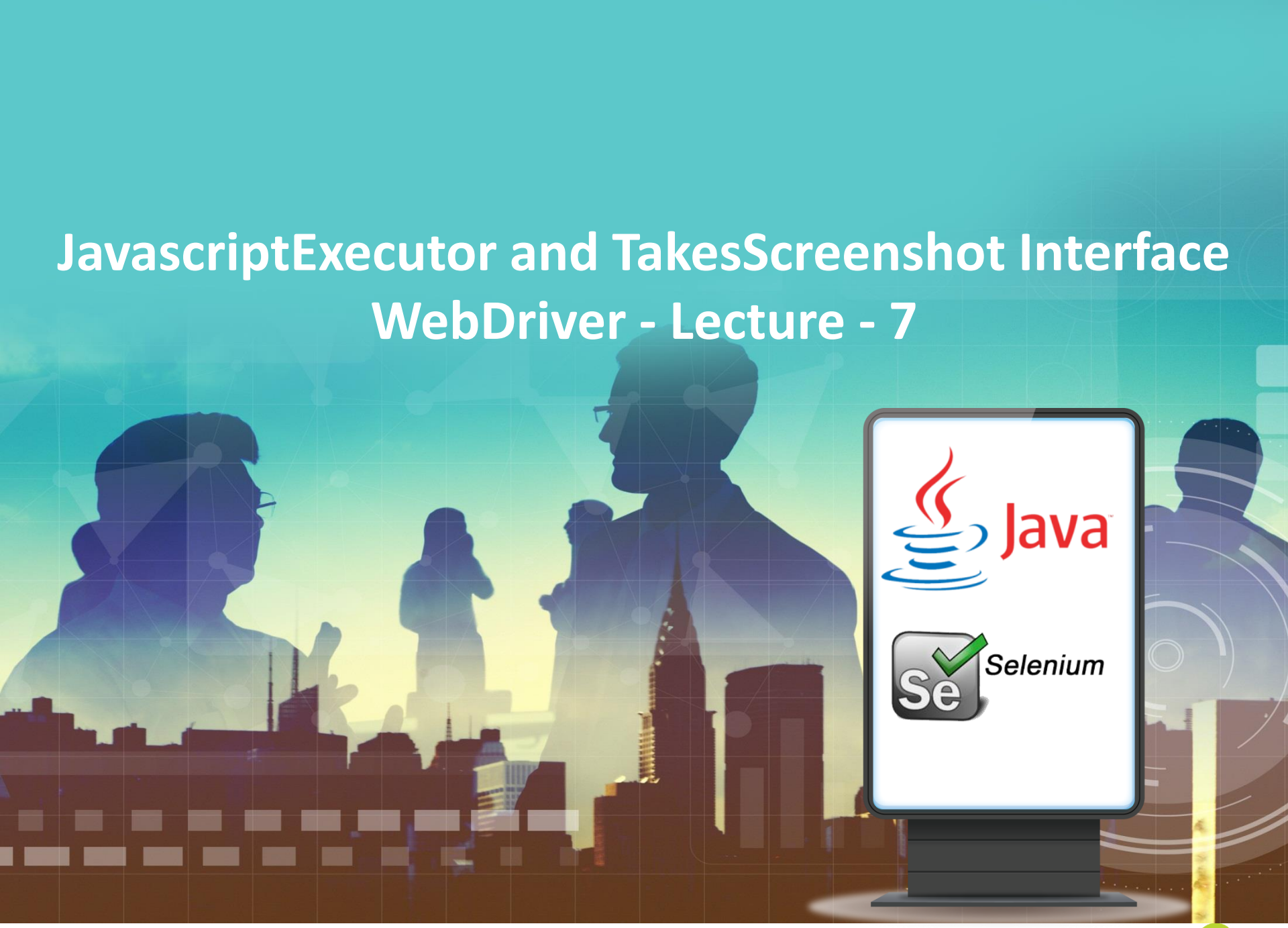
JavascriptExecutor and TakesScreenshot Interface WebDriver - Lecture - 7



Agenda

- Cookie Class in WebDriver
- Deleting All Cookies
- WebElement's Attribute And CSS Value
- JavascriptExecutor Interface
- Javascript Enter Text And Element Click
- Javascript Scroll to WebElement
- Javascript to get All WebElement's CSS
- TakesScreenshot Interface
- Save Screenshot at Project Location
- EventFiringWebDriver
- Quiz
- Homework

Cookie Class in WebDriver

- A **HTTP cookie** is used to store user information and their preferences.
  - It stores in a **key-value pair**.
  - It is a small piece of data sent from **Web Application** and stored in **Web Browser.**
  - Every time the user loads the website, the browser **sends the cookie back** to the server to notify the website of the user's previous activity.
- **Options Interface (inner interface**) in the **Webdriver interface** provides the methods like adding a cookie, deleting, retrieving a cookie in **Selenium Webdriver**.
- **Cookie class** is present in "**org.openqa.selenium**" package and can be used to store some **important information** like user payment Information, order confirmation etc.
- We can use the **addCookie(Cookie ck)** method to add a specific cookie to the **browser session**.
  Cookie newck = new Cookie("user1", "qamentor");
  driver.manage().addCookie(newck);
- We can retrieve the cookie added in browser session using **getCookieNamed(String name)** method:
  Cookie newC = driver.manage().getCookieNamed("user1");
  System.out.println(newC.getName()+"*******"+newC.getValue());***

```
driver.manage().getCookies();                  // Return The List of all Cookies
driver.manage().getCookieNamed(arg0);          //Return a cookie according to name
driver.manage().addCookie(arg0);               //Create and add the cookie
driver.manage().deleteCookie(arg0);            // Delete specific cookie
driver.manage().deleteCookieNamed(arg0);       // Delete a cookie according to name
driver.manage().deleteAllCookies();            // Delete all cookies
```

Deleting All Cookies

- **deleteAllCookies()** method of **Options interface** deletes all cookies from browser session.
- **deleteAllCookies()** method can be used to **delete all preferences** of previous user and make browser session completely new for new user.
- Performing **deleteAllCookies()** after successful login in the app does **logout** and refreshing the browser opens the login screen.

```
WebElement userField = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("login-email")));
userField.sendKeys("qacult.demo@gmail.com");
wait.until(ExpectedConditions.presenceOfElementLocated(By.id("login-submit"))).click();
wait.until(ExpectedConditions.elementToBeClickable(By.id("password-login"))).sendKeys("testing123");
wait.until(ExpectedConditions.presenceOfElementLocated(By.id("login-submit-password"))).click();

Thread.sleep(3000);
driver.manage().deleteAllCookies();
driver.get("http://app.box.com");
```

This will delete all cookies from Browser session and logout.

WebElement's Attribute And CSS Value

- In addition to previous **WebElement's** Methods, two more methods are needed to be discussed:
- **String getAttribute(String name)** - Get the value of the given **attribute** of the element:
  - Will return the current value, even if this has been modified after the page has been loaded
  - In HTML code whatever is present in the left side of **'='** is an attribute name and on the right side is an attribute value
  - **getAttibute()** returns **'null'** if there is no such attribute

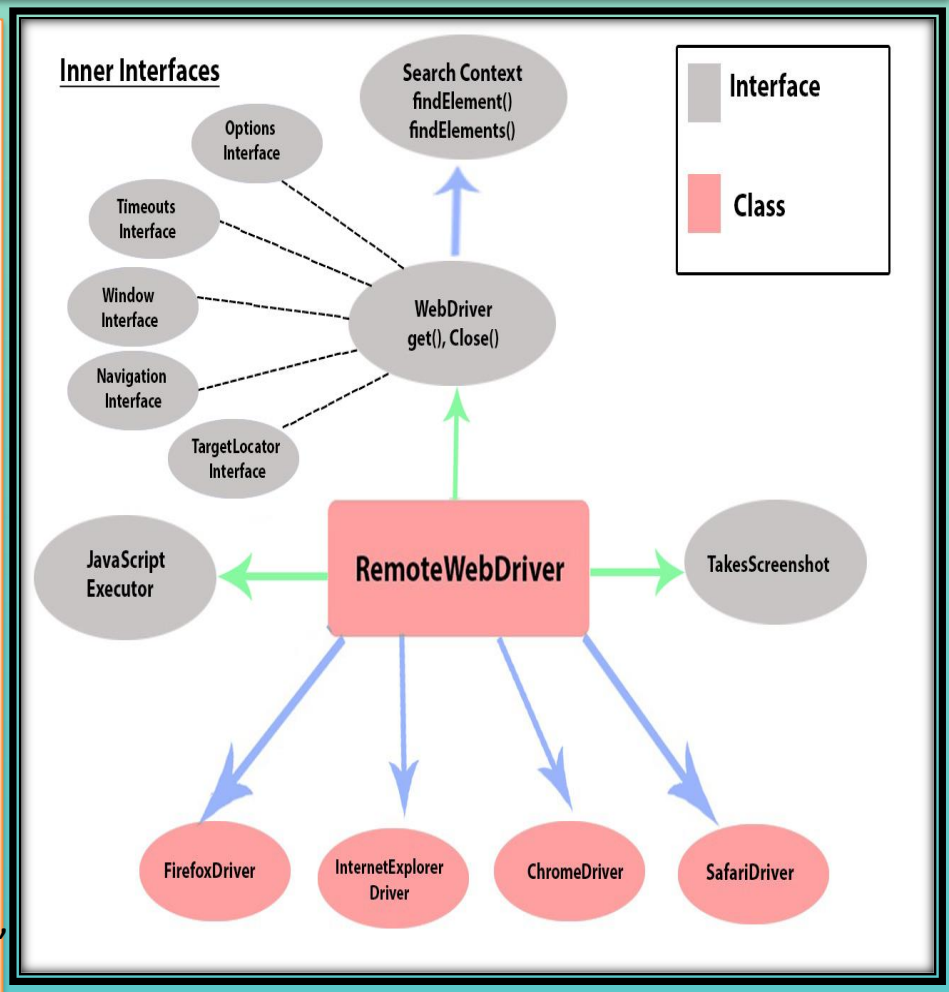
- **String getCssValue(String propertyName)** - Get the value of a given **CSS property** of **WebElement**
  - We have to pass **CSS property name**, which selenium have to fetch
  - Returns the current **computed value** of the CSS property
  - Returns **blank**, if there is no such CSS

```
System.out.println("The CSS value is: "+descPaste.getCssValue("font-size"));          //The CSS value is: 12.87px

System.out.println("The value of type attribute is: "+descPaste.getAttribute("type"));  //The value of type attribute is: text
```

JavascriptExecutor Interface

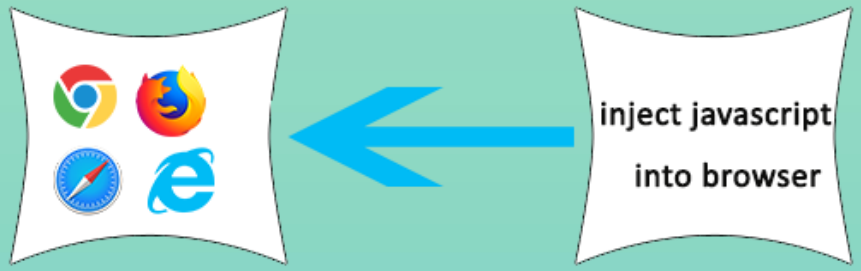
- **JavaScript** is the preferred language inside the browser to interact with **HTML DOM**
- **JavascriptExecutor** is an **Interface** that helps to execute **JavaScript** through **Selenium Webdriver**.

- **JavascriptExecutor Interface** provides two methods to run **javascript** on the selected window:
  - **Object executeScript(String script, Object... args)**
  - **Object executeAsyncScript(String script, Object... args)**
- Both of these methods are used to execute **javascript** and has **2** parameters:
  - ***Script*** – The JavaScript to execute
  - ***Arguments(args)*** –The arguments to the script are optional and may be empty.
- **arguments keyword** is javascript function build-in feature. You can use it to get real passed-in arguments when function is called.
  - We can use **multiple arguments** in our script just as an array and give their value through code like **arguments[0], arguments[1]** etc
- **Return Type** –It can be one of Boolean, Long, String, List, WebElement, or null.



Javascript Enter Text And Element Click

There is a need to **Class Cast WebDriver reference** to use **JavascriptExecutor** methods because **WebDriver interface** does not have JavascriptExecutor methods whereas **FirefoxDriver** or **ChromeDriver** class have them.



Javascipt to navigate to an URL

```
WebDriver driver = new FirefoxDriver();
JavascriptExecutor executor =(JavascriptExecutor)driver;
executor.executeScript("window.location.href = 'https://app.box.com';");
```

Click on a web element using javascript only having element id "login-email"

```
executor.executeScript("document.getElementById(\"login-email\").click();");
```

Click on a webelement using javascript and sending first argument as WebElement

```
WebElement emailTextField = driver.findElement(By.id("login-email"));
executor.executeScript("arguments[0].click();",emailTextField);
```

Sending text to webelement using javascript and sending first argument as WebElement and second argument as String

```
executor.executeScript("arguments[0].value=arguments[1]",emailTextField,"qacult");
```

Javascript Scroll to WebElement

- **To scroll down the web page by pixel**

  JavascriptExecutor js = (JavascriptExecutor) driver;

  js.executeScript("window.scrollBy(0,100000)");

- Javascript method **scrollBy(),** scrolls the web page by the specific number of pixels.The syntax of **scrollBy()** method in javascript is :

  executeScript("window.scrollBy(x-pixels,y-pixels)");

- **To scroll down the web page by the visibility of the element**
- Javascript method **scrollIntoView()** scrolls the page until the mentioned element is in **full view** :

  WebElement element = driver.findElement(By.linkText("Linux"));

  js.executeScript("arguments[0].scrollIntoView();", element);

- To scroll down the web page at the bottom of the page

  js.executeScript("window.scrollTo(0, document.body.scrollHeight)");

```
WebElement element1 = driver.findElement(By.cssSelector("a[id='pnnext']"));
((JavascriptExecutor) driver).executeScript("arguments[0].scrollIntoView(true);", element1, "scroll");
```

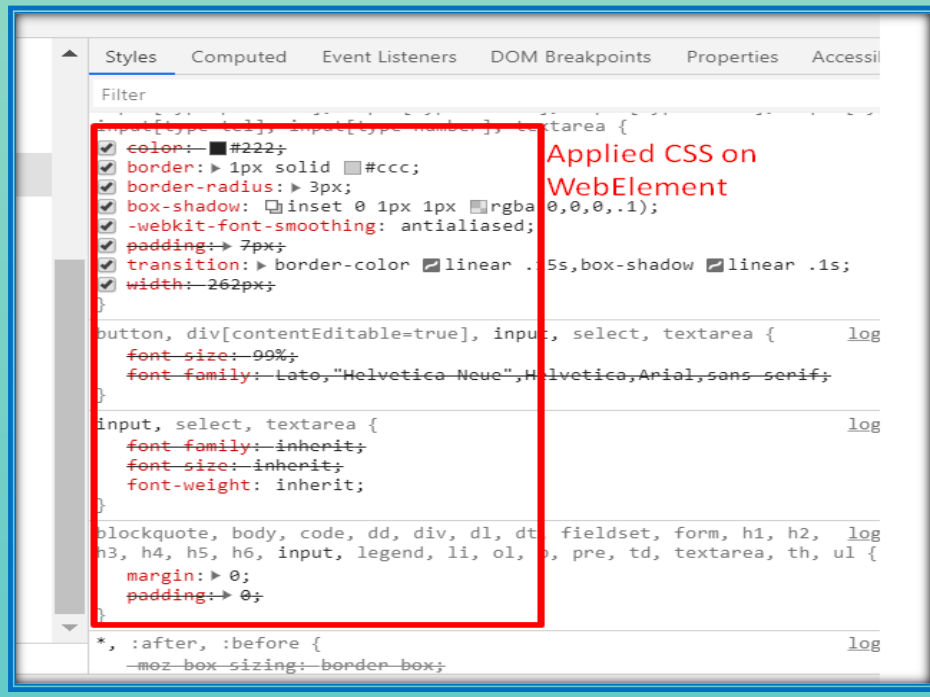
Javascript to get All WebElement's CSS

Get content height and content width of web page

```
int contentHeight = ((Number) ((JavascriptExecutor)driver).executeScript("return window.innerHeight")).intValue();
int contentWidth = ((Number) ((JavascriptExecutor)driver).executeScript("return window.innerWidth")).intValue();
```

Get all CSS attributes on WebElement

```
WebElement web = driver.findElement(By.tagName("div"));

JavascriptExecutor executor = (JavascriptExecutor)driver;

String script = "var s = '';" +

        "var o = getComputedStyle(arguments[0]);" +

        "for(var i = 0; i < o.length; i++){" +

        "s+=o[i] + ':' + o.getPropertyValue(o[i])+';';}" +

        "return s;";

Object allCss = executor.executeScript(script, web);

System.out.println(allCss);0
```



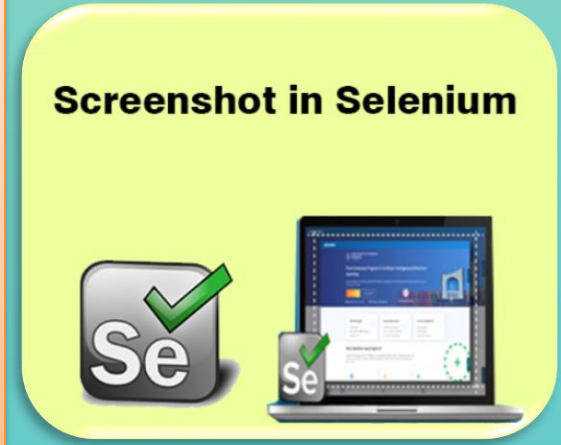
TakesScreenshot Interface

- **TakesScreenshot** Interface helps in taking the screenshot during the execution of automation script.
- **WebElement Interface** extends **TakesScreenshot** and all **browser driver** classes like **ChromeDriver, FirefoxDriver, InternetExplorerDriver, EdgeDriver** etc implements **TakesScreenshot** Interface through **RemoteWebDriver** class
- **TakesScreenshot** interface has a method named "**getScreenshotAs**" which can capture a **screenshot** and store it in different ways in specified locations.

- There is a need to **Class Cast WebDriver reference** to use **TakesScreenshot** method because **WebDriver interface** does not have **TakesScreenshot** method whereas **FirefoxDriver** or **ChromeDriver** class have them.
- Method For capturing screenshot:

  X getScreenshotAs(OutputType<X> target): Here the return type of method (X) depends on **output type target**.

  - If output type target is **BASE64**, then return type of method is **String**
  - If output type target is **BYTES**, then return type of method is **byte[]**
  - If output type target is **FILE**, then return type of method is **File**
  - If output type target is **class**, then return type of method is **Class<org.openqa.selenium.OutputType>**



Class casting of WebDriver object.

```
File srcFile = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
```

Save Screenshot at Project Location

- In Java, you can determine the current directory where your Java application is started in, using **System** class **getProperty()** method and ”**user.dir”** property

```
String currentDirectory = System.getProperty("user.dir");
```

- Steps for saving the **screenshot** at project’s location:
  - Create a String with random name using **currentTimeMillis()** method which will always return unique number for **current time in milliseconds** at the current project location using “**user.dir**” property.
  - Generate a screenshot with **output type as File**.
  - With “**org.apache.commons.io.FileUtils**” class and its **copyFile()** method, save the screenshot in your project’s desired location.

```
String screenshotPath = System.getProperty("user.dir")+"/screenshots/screenshot_"+System.currentTimeMillis() + ".png";
File srcFile = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
System.out.println("Failed and Screen capture saved at location "+ screenshotPath);

    try {
        FileUtils.copyFile(srcFile, new File(screenshotPath));
    } catch (IOException e) {

        System.out.println(e.getMessage());
    }
```

EventFiringWebDriver

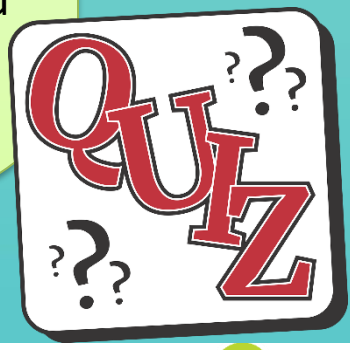
- **EventFiringWebDriver** is an **API** for **tracking** and **logging** the various events that happen when test scripts are executed using **WebDriver**.
- Many navigation events get fired **before and after a WebDriver internal event** occurs (such as before and after **navigating to a URL**, before and after **browser back-navigation**, before and after **findBy**, before and after **click** etc), and these can be **tracked** and **captured**.
- For capturing **WebDriver events**, you have to use:
  - **EventFiringWebDriver** class which is used to wrap **WebDriver** around to throw events
  - **WebDriverEventListner** *interface* that you have to implement to catch the **WebDriver** events
- **EventFiringWebDriver** class has a method register() which let you *register* our implementation of *WebDriverEventListner* to listen to the WebDriver *events.*

```java
public class EventHandler implements WebDriverEventListener{
    public void afterChangeValueOf(WebElement arg0, WebDriver arg1) {
        System.out.println("inside method afterChangeValueOf on " + arg0.toString());
    }
    public void afterClickOn(WebElement arg0, WebDriver arg1) {
        System.out.println("inside method afterClickOn on " + arg0.toString());
    }
    public void afterFindBy(By arg0, WebElement arg1, WebDriver arg2) {
        // TODO Auto-generated method stub
        System.out.println("Find happened on " + arg1.toString() + " Using method " + arg0.toString());
    }
    public void afterNavigateBack(WebDriver arg0) {
        System.out.println("Inside the after navigateback to " + arg0.getCurrentUrl());
    }
    public void afterNavigateForward(WebDriver arg0) {
        System.out.println("Inside the afterNavigateForward to " + arg0.getCurrentUrl());
    }
    public void afterNavigateTo(String arg0, WebDriver arg1) {
        // TODO Auto-generated method stub
        System.out.println("Inside the afterNavigateTo to " + arg0);
    }
    public void afterScript(String arg0, WebDriver arg1) {
        System.out.println("Inside the afterScript to, Script is " + arg0);
    }
    public void beforeChangeValueOf(WebElement arg0, WebDriver arg1) {
```

```java
public class programMain {

    public static void main(String[] args) throws InterruptedException {
        // TODO Auto-generated method stub

         WebDriver driver = new ChromeDriver();
         EventFiringWebDriver eventDriver = new EventFiringWebDriver(driver);

        EventHandler handler = new EventHandler();
        eventDriver.register(handler);

        eventDriver.get("https://app.box.com");
        Thread.sleep(2000);
        WebElement element = eventDriver.findElement(By.name("login"));
        element.click();

        element.sendKeys("qacult.demo@gmail.com");


    }

}
```

Quiz

- How will you add a **new cookie** to a browser session?
- Which **inner interface of WebDriver** have methods for adding or deleting cookies?
- Can you name **WebElement's** methods for getting their **attribute value** and **css value**?
- How do **BrowserDriver** classes implement **JavascriptExecutor Interface**?
- What does **executeScript()** method of **JavascriptExecutor** returns?
- Why is there a need to **cast WebDriver instance** in order to use **JavascriptExecutor** methods?
- Which **keyword** is used as a variable name in **javascript** to get its code value from **Java**?
- Can you name the method of **TakesScreenshot interface** used for taking screenshot and how it can be used?
- For **tracking WebDriver events**, which **class** and which **interface** is used and what is their purpose?



Homework

- Write a **WebDriver** script and
  - Navigate to https://amazon.com
  - **Scroll** to the bottom of the page through **javascript**
  - **Scroll** to top of page through **javascript**
  - **Scroll** to element with text "**Conditions of use**"
  - Take **Screenshot** of the bottom of the page.
  - Save screenshot in "**screenshot**" folder within the project.

- Use **EventFiringWebDriver** class for capturing and logging **WebDriver events** and create **WebDriver** script for **logging into Box app**.